

Module 1

Introduction to Reinforcement Learning

The primary goal of the presented modules on reinforcement learning (RL) design projects is enhancement of existing curricula and the development of new curricula on RL for undergraduate and graduate students in engineering. The designed projects will also benefit undergraduate and graduate students and instructors, in computer science, sciences, economy, business, and operational research. The RL projects, exercises, and homework problems presented in these modules come from all areas of engineering with emphasis on real physical systems such as aircraft, spacecraft, drones, robots, vehicles, green energy systems, bioengineering, and in general diverse dynamic systems encountered in industry.

In the past fifteen years RL has become an important engineering teaching and research area. Several universities have started offering graduate and undergraduate courses on RL. The projects are implemented in MATLAB so that practical working knowledge of MATLAB and Simulink is expected from the readers. The modules provide variety of RL design projects with their solutions using MATLAB's live scripts and functions, and Simulink.

Each designed project has an independent and self contained very brief presentation of analytical results on the subject under consideration, and with detailed RL design project formulations. We encourage students and instructors to master MATLAB and relevant MATLAB toolboxes by visiting the MathWorks website and/or to take the MathWorks abbreviated courses: "MATLAB Onramp", "Simulink Onramp" and "Reinforcement Learning Onramp". In Module 2, we will provide a short Simulink Manual and a brief summary of the main MATLAB functions from the Control System Toolbox relevant to the RL projects designed.

Our plan is to develop a set of MATLAB, Simulink, and Control System toolbox oriented *design projects, exercises, and homework problems* that will make RL easily understandable and simpler for using by a broad educational community, first of all students in all engineering fields. The developed material is based on the engineering approach to RL from the feedback control systems perspective, which in fact represents iterative, adaptive, and optimal feedback control of dynamic systems. The central theme evolves around the *approximate dynamic programming* techniques. The controls and systems approach is used to generate learning policies.

Our approach is based on optimal feedback control systems and Bellman's optimality principle [1]. Not surprising, Sutton and Barto (2020) in their well-known textbook [2] wrote: "We formalize the problem of reinforcement learning using ideas from dynamic systems theory, specifically, as the optimal control of incompletely-known Markov decision processes." and "One of the most pressing areas for future reinforcement learning research is to adopt and extend methods developed in control engineering with the goal of making it acceptably safe to fully embedded reinforcement learning agents into physical environments".

The three pillars of modern machine learning (ML) are supervised learning, unsupervised learning, and reinforcement learning (RL). RL means learning from interactions. RL techniques are widely used nowadays in many applications, and with high expectation to solve future challenging problems of artificial intelligence (AI). This expectation is due to specifics of RL that it is much closer the way humans think and perform tasks than neural networks based supervised and unsupervised learning methods, as pointed out in the well-known textbook by Sutton and Barto [2]: “Of all the forms of machine learning, reinforcement learning is the closest to the kind of learning that humans and other animals do, and many of the core algorithms of reinforcement learning were originally inspired by biological learning systems.” Moreover, the same authors in the same book wrote: “Reinforcement Learning has gradually become one of the most active research areas in machine learning, artificial intelligence, and neural network research.”

The designed projects are mostly based on iterative linear *optimal feedback control of dynamic systems*. The knowledge of feedback systems is important, but not essential to successfully master all design steps and methodology of presented projects and fully understand their solutions. *Solid mathematical background is required*, such as knowledge of linear algebra, differential and difference equations, some knowledge of dynamic systems and controls. The projects designed are introductory in nature, and advanced senior undergraduate students in engineering with strong mathematical background will be able to master successfully most of the presented material.

The approach taken is deterministic, which is specific to the engineering approach to RL. Some schools and researchers, particularly in computer science, consider RL using stochastic problem formulations, especially Markov decision processes (MDP) and Monte Carlo (MC) methods. RL has been developing over many years using both deterministic and stochastic formulations.

Several *overview papers* have provided very comprehensive surveys of the development of RL within the past fifteen years, with emphasis on engineering applications to real physical systems [3]-[8]. There are at least two dozen of books on RL written within computer science and a dozen of books on control systems engineering approach to RL, see for example, [9]-[14].

1.1 RL Brief History, Achievements, and Relation to Optimal Control

In this section, we present some basic historical facts and provide some fundamental information regarding the RL methodology and its relation to optimal control of dynamic systems. RL history is pretty long with the first results dating back hundred years ago. RL originated in experimental and theoretical studies of Russian physiologist Ivan Pavlov on animal learning during the 1920s. Around the same time a mathematical discipline known as *calculus of variations* made first attempts to solve what is today known as the optimal control problem. After thirty or so years, researchers in calculus of variations produced the fundamental results for the solution of a general optimal control problem: optimize an integral functional (scalar function of vector variables, called also the performance criterion or cost or reward) along trajectories of a nonlinear dynamic system. The first solution to this dynamic optimization problem was derived by Richard Bellman in the first part of the 1950s using the technique developed by himself that he named dynamic programming. The results of Bellman were summarized and published in his famous 1957 book titled *Dynamic Programming* [1].

The field of RL is very rich due to its long history and participation of a large number of researchers from diverse fields. Due to its analytical and computational complexity and diversity, RL has been evolving slowly during the past seventy years. The development of RL is first of all based on the fundamental work of Bellman on dynamic programming (DP) and his Optimality Principle, [1]. Researchers in calculus of variations, around the same time had produced the solution to the general optimal control problem of dynamic systems, including the case of constrained optimization, that resulted in the well-known Minimum Principle of Pontryagin [15].

RL has gained its maturity within computer science in this century, and has become an important research and educational topic in engineering in the past fifteen years, with more engineering schools offering RL undergraduate and graduate classes every year, and with more and more engineering faculty being involved in research and teaching on RL.

The engineering approach to RL is based on approximate dynamic programming (ADP), also known as adaptive dynamic programming. It has been known since the 1960s that dynamic programming solves optimization problems of continuous- and discrete-time, for both deterministic and stochastic dynamic systems. However, DP originally suffered from analytical and computational difficulties so that it took long time, until the beginning of this century, when ADP was discovered, and when ADP has become a powerful RL technique.

RL in general represents a computational method for learning from feedback received information through interactions with environment (dynamic system). It can be model-based (when a mathematical model of environment exists) and model-free when no such a model is available. The approach taken in this book will be either *model-based* or *partially model-based* RL or completely *model-free* RL, with applications to engineering systems whose dynamics can be described either by differential or difference equations. *The central theme in all presented methods will be discovering the built-in mechanism (algorithm, recursive program, or a set of repeated consistent physical actions) that iteratively improves control strategies through RL schemes such that RL strategies converge to the optimal solutions (optimal control, optimal system state trajectories, optimal system performance) for considered engineering dynamic systems.*

RL researchers in computer science consider *RL as an approximate optimal control technique.* The title of the paper by computer science researchers, [16], is “Reinforcement Learning is Direct Adaptive Optimal Control.” In addition to *approximate dynamic programming*, RL is also known under several different names: *adaptive dynamic programming* or *heuristic programming* or *neuro-dynamic programming* or *iterative dynamic programming* or *incremental dynamic programming* or *forward dynamic programming*. The optimal control system approach to RL culminated in the past fifteen years in the works of Frank Lewis and his coworkers, who published a large number of journal papers, for example, [3]-[6], [12], [14], and references there in.

1.2 Computer Science Basic RL Techniques

In the 1980s, the temporal-difference methods [17]-[18] were developed, followed by the RL methods based on Markov Decision Processes (MDP), as presented by [19] and [20], which has been considered as the beginning of the modern approach to RL within the computer science field. Researchers in computer science consider that the Bellman’s book [1] and its Chapter XI on Markovian Decision Processes (MDP) as the beginning of the modern era of computational RL,

which together with the Monte Carlo search techniques and the use of Neural Networks (NN), constitute the foundation of computer science approach to RL [2], [7], [9], [21]. In addition to Chapter XI from [1], more fundamental results regarding to dynamic programming and MDP from that time came from [20]-[21].

The *temporal difference (TD) learning* method that originated in the dissertation of Sutton [17] and the *Q-learning* method that originated in the dissertation of Watkins [19] revolutionized the RL techniques within computer science. These days RL is considered *as* an approximate optimal control technique (ADP, [2]). The well-known textbook [2] states: “We formalize the problem of RL using ideas from dynamic systems theory, specifically, as the optimal control of incompletely-known Markov decision processes.” And, “One of the most pressing areas for future RL research is to adopt and extend methods developed in control engineering with the goal of making it acceptably safe to fully embed reinforcement learning agents into physical environments”.

The temporal difference learning and the Q-learning techniques can be represented in their basic forms as follows.

Temporal Difference (TD) Learning [18]-[19]: A value function $V(s)$ of state variables s , called the *state value function*, is optimized using the TD algorithm, whose original formulation is given by [1]:

$$V(s_k) \leftarrow V(s_k) + \alpha[V(s_{k+1}) - V(s_k)] = (1 - \alpha)V(s_k) + \alpha V(s_{k+1}) \quad (1.1)$$

where $V(s_k)$ is the value (or estimate) function at present state s_k , $V(s_{k+1})$ is the value (or estimate) function at the next state s_{k+1} , $V(s_{k+1}) - V(s_k)$ defines the *temporal difference*, k is the present time, $k + 1$ is the next time instant, α is the step size, often called the learning rate.

Q-Learning [20]-[21] includes the action (control) variable a so that the *state-action value function* denoted by $Q(s, a)$ is optimized using the algorithm:

$$Q(s_k, a_k) \leftarrow Q(s_k, a_k) + \alpha[Q(s_{k+1}, a_{k+1}) - Q(s_k, a_k)] \quad (11.2)$$

where, in addition to quantities defined in (1.1), a_k is the action (control) signal at time k , and a_{k+1} is the action (control) signal at time $k + 1$. Q-learning has also its stochastic Markov Decision Process formulation [2], [7], [9].

The original TD-learning method (1.1) and the Q-learning method (1.2) are one-step iterative schemes, both based on a simple strategy:

$$\text{NEW ESTIMATE} \leftarrow \text{OLD ESTIMATE} + \alpha[\text{ERROR ESTIMATE}] \quad (1.3)$$

Both TD-learning and Q-learning have several variants, especially the ones that incorporate expected values and discount factors. Their development was mostly intuitive. *These simple formulas, having the nature of Euler's approximation of the first derivative, can be related to the continuous- and discrete-time Hamiltonians of the optimal control problem*, more precisely, to the main results of the Bellman's dynamic programming in continuous- and discrete-time domains. Moreover, MATLAB (and many other publications) names the following variant (using MATLAB notation) of Q-learning as the “*Bellman Equation*”

$$\text{new}Q(s, a) = Q(s, a) + \alpha[R(s, a) + \gamma \max_{a'} Q'(s', a') - Q(s, a)] \quad (1.4)$$

In (1.4), $\text{new}Q(s, a)$ is the new estimate of the state-action value function, $Q(s, a)$ is the previous estimate of the value-action value function, α is the step size also called the learning rate, $R(s, a)$ is the reward for taking action a at state s , γ is the discount factor for future awards, $\max_{a'} Q'(s', a')$ is the maximum expected value from state s' . The term $R(s, a) + \gamma \max_{a'} Q'(s', a')$ represents the best new estimate of the state-action value function. It corresponds to $Q(s_{k+1}, a_{k+1})$ in (1.2), which indicates the equivalence of (1.2) and (1.4). The Bellman equation (1.4) has more practical structure than the original Q-learning formula (1.2) due to introduction of a more general reward function and the discount factor for future awards. Several variants of (1.4) can be found in the RL literature with some of them presented in [2].

As stated in the MATLAB Reinforcement Learning Toolbox Manual: “The Bellman equation allows the agent to solve the Q-table over time by breaking up the whole problem into multiple simpler steps. Rather than solving the true value of a state/action pair in one step, the agent will update the value each time a state/action pair is visited through dynamic programming. The Bellman equation is important for Q-learning as well as other learning algorithms, such as DQN [Deep Q-Network Learning].”

1.3 General Reinforcement Learning Block Diagram and MATLAB Live Editor

A general RL block diagram is essentially an expanded feedback control system block diagram as presented in Fig. 1.1, which in addition to the classical feedback loop from the state variables (lower feedback loop) has another (upper) feedback loop that provides information about the performance (cost, reward) at all times using information about the state variables.

MATLAB’s Live Editor environment is very convenient for implementation of the RL mechanism. In Module 2, we will briefly review features of the MATLAB Live Editor and corresponding live scripts and live functions, as well provide a sufficient introduction to Simulink. In this section, using the block diagram from Figure 1.1 we indicate how particular blocks of the RL mechanism can be implemented using MATLAB’s live scripts, live functions, and Simulink. It can be seen from Figure 1.1 that *the control agent can be implemented using a MATLAB live script (program). The state of the system upon given control input (action), obtained via a feedback channel measurement, can be represented by a MATLAB live function. The reward (cost, performance) estimation via the upper feedback loop can be obtained using a MATLAB live function. The environment (engineering system subjected to RL) can be modeled using Simulink (or in simpler cases using a live function).* Even more, in some simpler RL problems, live functions for particular blocks in Figure 1.1 can be replaced by (ordinary, common) MATLAB functions. Note that in the case of RL, the *reference input* (which in fact represents the system desired output at steady state and it is needed for the purpose of controlling the system output only) is *mostly set to zero*, so that the state feedback measurement directly enter the controller (agent) block. In some rare cases the reference input can contribute to the reward function and observations.

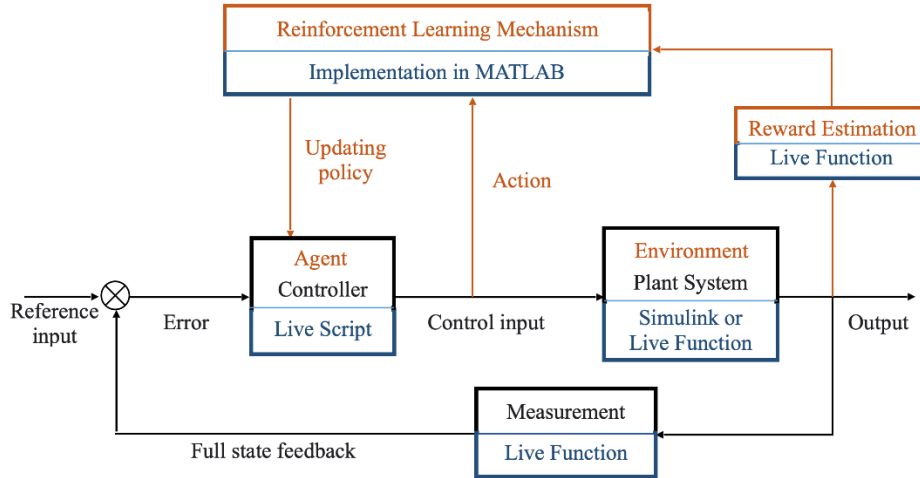


Figure 1.1: MATLAB implementation of the reinforcement learning mechanism

The MATLAB triad: live script, live function, Simulink will play essential roles in the developed RL design projects for engineering systems to be presented in the follow-up modules that correspond to different formulations: discrete, continuous, single or multiple agents, exactly or partially known environment (system) or even completely unknown environment that only reveals either its states or the approximate performance values via feedback signals. As a matter of fact, the MATLAB live scripts interactive capabilities in conjunction with Simulink modeling and simulation capabilities can create an excellent, software oriented testbed for verification of reinforcement learning algorithms for engineering systems.

It is the opinion of the authors of these modules that students, faculty, RL researchers, and practicing engineers do not need any specialized *Reinforcement Learning Manual for Engineering Dynamic Systems*. They need to master the MATLAB live editor with its live scripts and live functions, and Simulink's tremendous dynamic system modeling and simulation capabilities in order to be able to implement almost any RL algorithm dealing with real dynamic systems.

1.4 Engineering and Computer Science RL Analogies/Similarities

In the previous sections, we have outlined similarities and differences between the engineering and computer science (in general sciences) approaches to RL. In this section, we indicate their similarities by providing corresponding analogous quantities. First of all, graphically, using formulas and a flow chart, the RL process of successive interactions, paralleling iterative feedback control, can be symbolically represented as given in Table 1.1. Programming in MATLAB, using its interactive live scripts and live functions, provides excellent opportunities to implement RL strategies along the lines depicted in Figure 1.1. Simulink can be naturally used to model the environment, and establishment of the RL mechanism for a particular engineering system. RL main quantities have their analogs in sciences and engineering (optimal control of dynamic systems). They are summarized in Table 1.2, where the quantities used in sciences are given in the left column and those in engineering are presented in the right column.

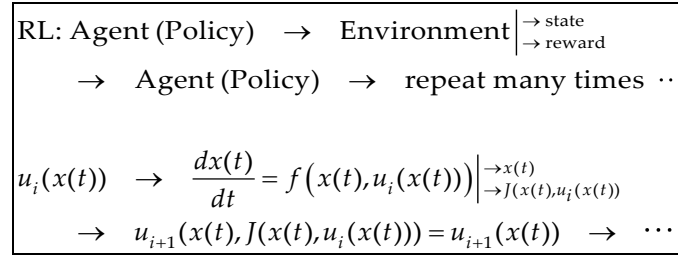


Table 1.1 Reinforcement learning as a repetitive feedback control technique

Sciences Reinforcement Learning	Engineering Optimal Control Systems
$s(t)$ state of an abstract system	$x(t)$ state of a dynamic system
$a(t)$ - agent or actor action	$u(t)$ - control input
$r(t)$ - scalar reward	$J(x(t), u(t))$ - scalar performance criterion
Environment: in general it is an abstract space. It can be a state of a game, a subset of R^n , or anything else that changes in time.	R^n - state space, with ODE system dynamics $\frac{dx(t)}{dt} = f(x(t), u(t)), \quad y(t) = g(x(t), u(t))$ or a dynamic system described by PDEs

Table 1.2 Reinforcement learning and optimal control

1.5 Organization of Modules

The first two modules are introductory, presenting an introduction to RL and introduction to Simulink and MATLAB's basic RL functions and its Live Editor. The remaining modules are organized into various types of design projects, each presented as an independent entity. In addition to the elementary RL theory on a particular RL topic, basic MATLAB codes and standard demos, are provided. The modules are organized as follows:

PART I: INTRODUCTION

Module 1 : Introduction to RL with comparison of computer science RL and engineering RL.

Module 2: Short Simulink manual and some functions from Control System Toolbox important for RL, including an introduction to MATLAB live scripts and live functions.

PART II: MODEL BASED REINFORCERMENT LEARNING

Module 3: Introduction to dynamic programming in discrete-time. The Hamiltonian and the Q-function. Discrete-time linear-quadratic (LQ) optimal control and the Hwuer algorithm.

Module 4: Introduction to dynamic programming in continuous time. LQ optimal control and the algorithm of Kleinman. RL for affine nonlinear optimal control systems.

Module 5: Continuous- and discrete-time policy iterations and value iterations.

Module 6: RL via policy iterations for zero-sum differential games.

Module 7: RL for Nash differential games.

PART III: MODEL FREE REINFORCERMENT LEARNING

Module 8: RL for partially model free dynamic systems.

Module 9: RL for completely model free reinforcement learning.

Module 10: Model free RL for zero-sum differential games.

Module 11: Model free RL for Nash differential games.

Appendix: It will contain solutions to some MATLAB/Simulink based homework problems, solutions to some design projects. It will be available to the class instructors.

The developed projects will be posted on Github and Professor Gajic's website for a public use, and be available to the MathWorks company for inclusion in any of their products.

1.6 References

- [1] Bellman, R., *Dynamic Programming*, Princeton University Press, Princeton NJ, 1957. Republished by Dover Publications Inc., Mineola NY, 2003.
- [2] Sutton, R. and A. Barto, *Reinforcement Learning: An Introduction*, MIT Press, Cambridge, MA, 2020.
- [3] Lewis, F. and D. Vrabie, "Reinforcement learning and adaptive dynamic programming for feedback control," *IEEE Circuits and Systems Magazine*, 32-50, Third Quarter, 2009.
- [4] Lewis, F., D. Vrabie, and K. Vamvoudakis, "Reinforcement learning and feedback control," *IEEE Control Systems Magazine*, 76-105, 2012.
- [5] Vamvoudakis, K., H. Modares, B. Kiumarsi, and F. Lewis, "Game theory-based control system algorithms with real-time reinforcement learning: How to solve multiplayer games on line," *IEEE Control Systems Magazine*, 33-52, 2017.
- [6] Kiumarsi, B., K. Vamvoudakis, H. Modares, and F.L. Lewis, "Optimal and autonomous control using reinforcement learning: A survey," *IEEE Transactions Neural Networks and Learning Systems*, Vol. 29, 2042-2061, 2018.
- [7] Vidyasagar, M., "A tutorial introduction to reinforcement learning," *SICE Journal of Control, Measurement, and Transactions on Neural Networks and Learning Systems*, Vol. 35, 5064-5078, 2024. *System Integration*, Vol. 16, 172-191, 2023.
- [8] Wang, X., S. Wang, X. Liang, D Zhao, J. Huang, and X. Xu, "Deep reinforcement learning: A Survey," *IEEE*
- [9] Bertsekas, D. and J. Tsitsiklis, *Neuro-Dynamic Programming*, Athena Scientific, Nahua, NH, 1996.
- [10] Bertsekas, D., *Reinforcement Learning and Optimal Control*, Athena Scientific, Belmont, MA, 2019.
- [11] Bertsekas, D., *A Course in Reinforcement Learning*, Athena Scientific, Belmont, MA, 2023.
- [12] Vrabie, D., K. Vamvoudakis, and F. Lewis, *Optimal Adaptive Control and Differential Games by Reinforcement Learning Principles*, IET, London, United Kingdom, 2013.
- [13] Meyn, S., *Control Systems and Reinforcement Learning*, Cambridge University Press, 2022.
- [14] Lian, B., W. Xue, F. Lewis, H. Modares, and B. Kiumarsi, *Integral and Inverse Reinforcement Learning for Optimal Control Systems and Games*, Springer, 2024.
- [15] Pontryagin, L., V. Boltyanskii, R. Gamkrelidze, and E. Mischenko, *The Mathematical Theory of Optimal Processes*, Interscience, New York, 1962.
- [16] Sutton, R., A. Barto, and R. Williams, "Reinforcement learning is direct adaptive optimal control," *IEEE Control Systems Magazine*, Vol. 12, 19-22, 1992.
- [17] Sutton, R., *Temporal Credit Assignment in Reinforcement Learning*, Ph. D. Dissertation, University of Massachusetts, Amherst, MA, 1984.
- [18] Sutton, R., "Learning to predict by the methods of temporal differences," *Machine Learning*, Vol. 3, 9-44, 1988.
- [19] Watkins, C. *Learning from Delayed Rewards*, Ph. D. Dissertation, University of Cambridge, MA, 1989.
- [20] Watkins, C. and P. Dayan, "Q-Learning," *Machine Learning*, Vol. 8, 279-292, 1992.
- [21] Graesser, L. and W. Keng, *Foundations of Deep Reinforcement Learning*, Addison Wesley, 2020.
- [22] Murray, J., C. Cox, G. Lendaris, and R. Sacks, "Adaptive dynamic programming," *IEEE Transactions on Systems, Man, and Cybernetics*, Vol. 32, 140-153, 2002.